✅ Menu Driven Queue Program in C (Array Based)

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

// Insert (Enqueue)
void insert() {
    int value;
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    printf("Enter value to insert: ");
    scanf("%d", &amp;value);

    if (front == -1)
        front = 0;

    rear++;
    queue[rear] = value;

    printf("%d inserted into queue\n", value);
}

// Delete (Dequeue)
void delete() {
    if (front == -1 || front &gt; rear) {
        printf("Queue Underflow\n");
        return;
    }

    printf("%d deleted from queue\n", queue[front]);
    front++;
```

```
    // Reset queue when empty
    if (front &gt; rear) {
        front = rear = -1;
    }
}

// Display
void display() {
    if (front == -1) {
        printf("Queue is Empty\n");
        return;
    }

    printf("Queue elements are: ");
    for (int i = front; i &lt;= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
    int choice;

    while (1) {
        printf("\n--- Queue Menu ---\n");
        printf("1. Insert an element\n");
        printf("2. Delete an element\n");
        printf("3. Display the queue\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &amp;choice);

        switch (choice) {
            case 1:
                insert();
                break;
            case 2:
```

```
                delete();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting...\n");
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}
```

Types of Queues

- **Circular Queue**

Algorithm for Dequeue Operation

```
Procedure Dequeue
    If queue is empty then
        return UNDERFLOW
    End If

    data = queue[front]
    front = front + 1

    return data
End Procedure
```

Example of Dequeue Operation (C Code)

```
int dequeue()
{
    if (isempty())
```

```
        return -1;   // Underflow condition

    int data = queue[front];
    front = front + 1;

    return data;
}
```

All operations in a queue:-

```
#include <stdio.h>
#define MAX 5

int cq[MAX];
int front = -1, rear = -1;

int isFull() {
    return (front == (rear + 1) % MAX);
}

int isEmpty() {
    return (front == -1);
}

void enqueue(int value) {
    if (isFull()) {
        printf("Circular Queue Overflow\n");
        return;
    }

    if (isEmpty()) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % MAX;
    }
```

```
    cq[rear] = value;
    printf("%d inserted into circular queue\n", value);
}

void dequeue() {
    if (isEmpty()) {
        printf("Circular Queue Underflow\n");
        return;
    }

    int deleted = cq[front];
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX;
  }

    printf("%d deleted from circular queue\n", deleted);
}

void peek() {
    if (isEmpty()) {
        printf("Circular Queue is Empty\n");
        return;
    }

    printf("Front element is %d\n", cq[front]);
}

void display() {
    if (isEmpty()) {
        printf("Circular Queue is Empty\n");
        return;
    }

    int i = front;
    printf("Circular Queue elements are: ");
    while (1) {
        printf("%d ", cq[i]);
```

```
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

void size() {
    if (isEmpty()) {
        printf("Size of circular queue: 0\n");
        return;
    }

    int count = (rear - front + MAX) % MAX + 1;
    printf("Size of circular queue: %d\n", count);
}

int main() {
    int choice, value;

    while (1) {
        printf("\n--- Circular Queue Menu ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Check if Full\n");
        printf("6. Check if Empty\n");
        printf("7. Size\n");
        printf("8. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &amp;choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &amp;value);
                enqueue(value);
                break;
```

```
        case 2:
            dequeue();
            break;

        case 3:
            peek();
            break;

        case 4:
            display();
            break;

        case 5:
            if (isFull())
                printf("Circular Queue is Full\n");
            else
                printf("Circular Queue is Not Full\n");
            break;

        case 6:
            if (isEmpty())
                printf("Circular Queue is Empty\n");
            else
                printf("Circular Queue is Not Empty\n");
            break;

        case 7:
            size();
            break;

        case 8:
            printf("Exiting...\n");
            return 0;

        default:
            printf("Invalid choice\n");
}
    }
```

```
    return 0;
}
```

This program includes all operations:

- Enqueue
- Dequeue
- Peek
- Display
- isFull
- isEmpty
- Size
- Exit

To run:

- Save as `circular_queue.c`
- Compile: `gcc circular_queue.c -o circular_queue`
- Run: `./circular_queue`

Time Complexity of Queue Operations

Operation	Time Complexity
Enqueue	O(1)
Dequeue	O(1)
Front (Peek)	O(1)
isEmpty	O(1)
Size	O(1)

👉 All operations in a queue take **constant time.**

Queue: Task Scheduling (Real-Time Application)

- Queues are widely used in **CPU scheduling** in operating systems.

Flow:

- **Job Queue → Ready Queue → CPU → Exit**
- If a process needs I/O:
  - It moves to **I/O Waiting Queue**
  - Then returns back to **Ready Queue**

👉 This ensures **efficient process management** using FIFO.

Breadth-First Search (BFS)

- BFS is a graph traversal algorithm that uses a **queue.**

Working:

1. Start from a node
2. Visit it and add neighbors to queue
3. Remove node from queue
4. Repeat until all nodes are visited

👉 BFS always explores **level by level**.

Buffer Management

- Queues are used in **buffer systems** to store temporary data.

👉 Example:

- Keyboard input buffer
- Data streams

👉 Helps in:

- Handling **data flow smoothly**
- Preventing **data loss**

Other Applications of Queues

- **Print Spooling** (printer queue)
- **Network Buffers**
- **Task Queues in Concurrent Programming**
- **Banking Systems (customers in line)**
- **Traffic Management Systems**

Common Mistakes and Pitfalls

- ❌ Not checking **queue underflow** (empty queue)
- ❌ Poor synchronization in **multithreading**
- ❌ Improper queue size management